WorkTrace: How Coding Agents Explore, Change Code, and Check Their Work

Anonymous working manuscript

Annotated research draft — 26 October 2026

Abstract

Coding agents inspect files, change code, and run checks. Final success and tool-call counts do not show how agents organize this work. We present WorkTrace, a four-stage framework for this analysis. It links each tool operation to a work target, an inferred code version, and earlier evidence. We study 6,300 runs from three models under one harness. We also study 6,000 runs from one model under three harnesses. The analysis shows two work patterns. First, Gemini performs more checks on the final code version than on earlier versions. This gap remains under three saved version rules. Second, OpenHands and SWE-agent revisit known targets more often than mini-swe-agent on the same tasks. Their mean revisit rates are 32.9%, 30.0%, and 18.2%, respectively. Exact-repeat rates are close to 0.2% for all three harnesses. These findings show where work accumulates and which system comparisons need closer inspection. They do not establish that the additional work is unnecessary. We explain how evidence about test scope and unresolved questions can support that next assessment.

Working-draft status. This revision adds statistical analyses of saved data. It adds no agent runs or human labels. Red notes identify the next research steps.

REVISION 01. Pass the seven audit regression cases, then rebuild version summaries. Publish changes in verification gaps and backtrack rates. Re-annotate operations whose evidence changed.

1 Introduction

Coding agents often solve the same task through different sequences of work. One agent may inspect many files before an edit. Another may edit early and then run many checks. A final patch and a tool-call count cannot explain this difference.

We need to know where the work occurs. Does an agent check each code version, or mainly the final version? Does a harness return to known targets, even when it rarely repeats an exact command and output? These questions help researchers compare systems and choose cases for closer analysis.

TRACE provides a useful approach to such questions (Zhang et al., 2026). It first measures broad behavior, then reconstructs reasoning graphs, and then studies common patterns. It connects those patterns to intermediate answer correctness and further utility tests. Coding-agent traces require a different representation. Code changes during a run, and two different commands can test the same code property.

We introduce WorkTrace to reconstruct this external work. The framework has four stages: collect operations, infer work relations, construct progression graphs, and compare graph patterns. Two linked views show target visits and code versions. Together, they connect exploration, code changes, and verification.

Our study has two comparison sets. The first compares three models on the same tasks under mini-swe-agent. The second compares three harnesses on the same tasks with Qwen fixed. The results support two findings:

- **Final-version checking.** Gemini places more checks on its final inferred code version than on an average earlier version. Luna and Terra show no similar positive gap.
- **Target revisits.** OpenHands and SWE-agent return to known targets more often than mini-swe-agent. This difference remains when we use the same rate definition and match tasks.

The framework makes these patterns measurable. It also identifies the evidence needed to interpret them. More checks can improve coverage. A file revisit can answer a new question. We therefore separate an observed work pattern from a claim that the work is excessive.

REVISION 02. Register the primary paired contrasts: verification gaps by model and revisit rates by harness. Freeze task splits before fresh-task confirmation.

2 Related Work

Work and reasoning structures. TRACE studies how proposed answers change during reasoning (Zhang et al., 2026). Bouzenia and Pradel study thought–action–result sequences in software agents (Bouzenia and Pradel, 2025). WorkTrace studies targets, code versions, and evidence within the same run. These links support comparisons of where agents spend their work.

Redundant work and efficiency. RedundancyBench evaluates redundant steps and their supporting evidence (Zhu et al., 2026). Its annotation procedures can inform our validation. AgentDiet reduces agent history (Xiao et al., 2026). TRIM studies unnecessary changes in final patches (Mathai et al., 2026). These studies address related questions, but each uses a different outcome. A repeated observation, a removable history item, and an unnecessary code change are not interchangeable labels.

Dependency and utility evaluation. CoTJudger constructs reasoning dependencies and validates short sufficient paths (Li et al., 2026). CAID separates local novelty from goal relevance and contextual progress. PACE tests its judgments through selective compression and pruning controls (Lee and Yu, 2026). These methods motivate the proposed evidence links and controlled contrasts in Section 5.3. Our extension must account for changing code state and justify actions from the information available at the time.

Interaction with users. SWE-TaskFlow studies task delivery (Slinko et al., 2026). Our comparisons use fixed instructions.

REVISION 03. Compare evidence judgments with exact repetition and the current target/version view. Include a state-blind ablation. A graph representation alone is not the contribution.

3 Benchmark Agent Work

We first compare final success and the amount of work. We then use WorkTrace to locate that work within each run.

3.1 Across Models

The first set contains 700 tasks, three models, and three runs per task and model. All 6,300 runs use mini-swe-agent. The saved operation table contains 188,609 operations. We retain the model names in the source metadata: Gemini 3.7 Flash, GPT-5.6 Luna, and GPT-5.6 Terra. Gemini has higher final success and uses more calls than either GPT model (Table 1). This comparison does not show whether its additional work is useful. The models and their released settings differ. They also differ in the number of code versions and checks, which we study in Section 5.

3.2 Across Harnesses

The second set uses Qwen3.8-27B from Open-SWE-Traces v1.2 (NVIDIA, n.d.). It contains 2,000 shared tasks with one selected run per harness. The three harnesses are mini-swe-agent, OpenHands, and SWE-agent. Together, the 6,000 runs contain 370,992 operations. OpenHands and SWE-agent use more calls than mini-swe-agent (Table 2). All three have exact-repeat rates near 0.2%. However, their target revisit rates differ. Section 5.1 examines this difference with paired task comparisons.

3.3 Reference-Patch Size

We also compare work across reference-patch size quartiles (Figure 1). Patch size is a proxy for task demands. It is not a direct measure of reasoning difficulty. Across the model set, pooled mean calls increase from 25.0 in Q1 to 34.4 in Q4. Mean success falls from 69.6% to 37.5%. Exact-repeat rates are 0.70%, 0.75%, 0.83%, and 0.75% across the four bins. These values do not support a steady increase in exact-repeat rates with patch size.

The harness profiles also separate more work from more exact repetition. All three harnesses use more calls on larger-patch tasks. Their revisit rates also rise between Q1 and Q4. These profiles motivate structural analysis, but do not establish excessive work. We keep the saved success-defined difficulty bins outside this comparison because their success trends follow from the bin definition.

3.4 Comparison Method

We analyze the model and harness sets separately because they contain different tasks. These sets cannot establish a model–harness interaction.

The additional analyses use 5,000 task-bootstrap samples with seed 20261026. Each sample draws tasks with replacement and keeps their associated runs together. Paired comparisons keep the same task on both sides. We report 95% percentile intervals. These are exploratory intervals, without adjustment for multiple comparisons.

For the version analysis, we require at least two inferred versions and a final version other than the initial repository state. We then select task–repeat pairs that meet this rule for every model under every version rule. This gives 892 matched pairs from 418 tasks, with 892 runs per model. This selected subset tests whether the pattern persists under common conditions. It does not represent every run.

REVISION 04. Classify the 4,198 missing results by operation kind and terminal status. Report paired inclusion counts and released settings for each system condition.

4 Framework for Work Progression

WorkTrace maps a tool sequence into a description of the work. Table 3 defines the main terms.

4.1 Stage 1: Collect Operations

We use saved tool traces from the two comparison sets. The parser records each call, its result, its position, and the available exit status. It assigns operation types such as read, search, edit, run, and submit.

The parser also splits shell chains to inspect their parts. The native call remains the measurement unit. A call can contain several action types. Normalization removes selected volatile fields from execution outputs. Exact-repeat detection compares the normalized command and output hash.

4.2 Stage 2: Infer Work Relations

For each operation, the framework retrieves two possible earlier matches. One is the latest operation on the same target. The other is the latest same-type operation with the same usable output. The current rules exclude short or explicitly truncated outputs from the second search.

Rule facts describe whether the output matches and whether code may have changed. An automatic annotator then labels the operation's role, relation, and contribution. Roles include exploration, revision, and verification. Relations include unmatched, revisit, reproduction, and

Model	Runs	Success (%)	Calls/run	Input tokens (M)	Exact repeats (%)
Gemini 3.7 Flash	2,100	60.0	53.1	3.34	2.21
GPT-5.6 Luna	2,100	50.0	17.6	0.25	0.02
GPT-5.6 Terra	2,100	54.2	19.0	0.24	0.05

Table 1: Saved cross-model benchmark on 700 matched tasks. Tokens and calls are per-run means. Exact repeats require the same normalized command and output. Their rates are means of per-run rates. Success means that the dataset marks the task as resolved.

Harness	Runs	Success (%)	Calls/run	Exact repeats (%)	Target revisits (%)
mini-swe-agent	2,000	57.1	51.8	0.20	18.2
OpenHands	2,000	51.0	66.2	0.18	32.9
SWE-agent	2,000	54.5	67.5	0.20	30.0

Table 2: Cross-harness benchmark with Qwen fixed. This revision calculates all rates as means of per-run rates. Older saved summaries used pooled operation rates for revisits. The new values permit a consistent comparison.

Term	Meaning in this paper
Operation	One native tool call and its recorded result.
Target	The file, command, or tool object that an operation addresses.
Revisit	An operation with an earlier same-target candidate.
Version	An inferred code state after a detected change.
Check	A run operation attached to a version, excluding detected read-only inspections.
Non-distinct	The annotator finds no new contribution relative to the cited earlier evidence.

Table 3: Terms describe recorded work. A check is not automatically a test, and a revisit is not automatically redundant.

uncertain. Contributions can be distinct, non-distinct, or uncertain.

The validator requires a cited match for a non-distinct label. Thus, the labels measure contribution within the retrieved evidence. They do not measure all unnecessary work. Appendix A reports the current annotation checks.

REVISION 05. In the pilot, allow references to any earlier operation. Report candidate-retrieval recall against independent valid matches. Separate uncertain context from genuinely unmatched work.

4.3 Stage 3: Construct Progression Graphs

Target graph. Each node represents a work target. Directed edges follow the operation sequence. Consecutive visits to one target form a self-loop. A later return forms a return edge. These edges locate repeated attention without assuming repeated information.

For example, an agent can read two different ranges of one file. The target repeats, but the second range can supply new evidence. Target identity also depends on the tool interface. A structured editor may expose a file path directly. A shell command may encode that path within a more specific expression.

Version graph. Each node represents an inferred code version. Detected code changes create new versions. Checks attach as self-loops to the version on which they run. Recognized restores can return to earlier versions.

A new node after a failed check receives the stored rule label `correct`. This label means revision after failure. It does not mean the resulting code is correct. Only final task success has a dataset outcome.

We retain three saved version rules. The first creates a version for every detected write, including uncertain writes. The second uses known writes only. The third excludes writes confined to inferred support files. Support files include temporary diagnostic scripts. The third rule uses final submission metadata, so it is a retrospective analysis.

REVISION 06. Use content identity for state merges and require successful restores. Track known changed paths and unknown side effects separately. Re-run the seven audit cases.

4.4 Stage 4: Compare Patterns

We group version graphs by model and version count. New versions advance horizontally. A new path after a backtrack moves to a higher row. Graphs align by these coordinates.

Node and edge weights record how often each component appears. The saved figures retain components present in at least 30% of grouped runs. We display groups with at least 20 runs. This procedure follows TRACE's coordinate-based aggregation approach (Zhang et al., 2026).

We also calculate measures before graph pruning. The verification gap compares final-version checks with the mean checks on earlier versions. The revisit rate divides operations with a same-target candidate by all operations in a run. These measures support direct comparisons beyond visual inspection.

REVISION 07. Replot common version groups at presence thresholds 0.1, 0.3, and 0.5. Report unpruned backtrack rates alongside the figures.

5 Findings

5.1 Work Progression Patterns

TRACE's Explorer and Late Landing patterns depend on intermediate answer correctness (Zhang et al., 2026). Our data lacks intermediate code-correctness labels. We therefore name our patterns by the work that we observe.

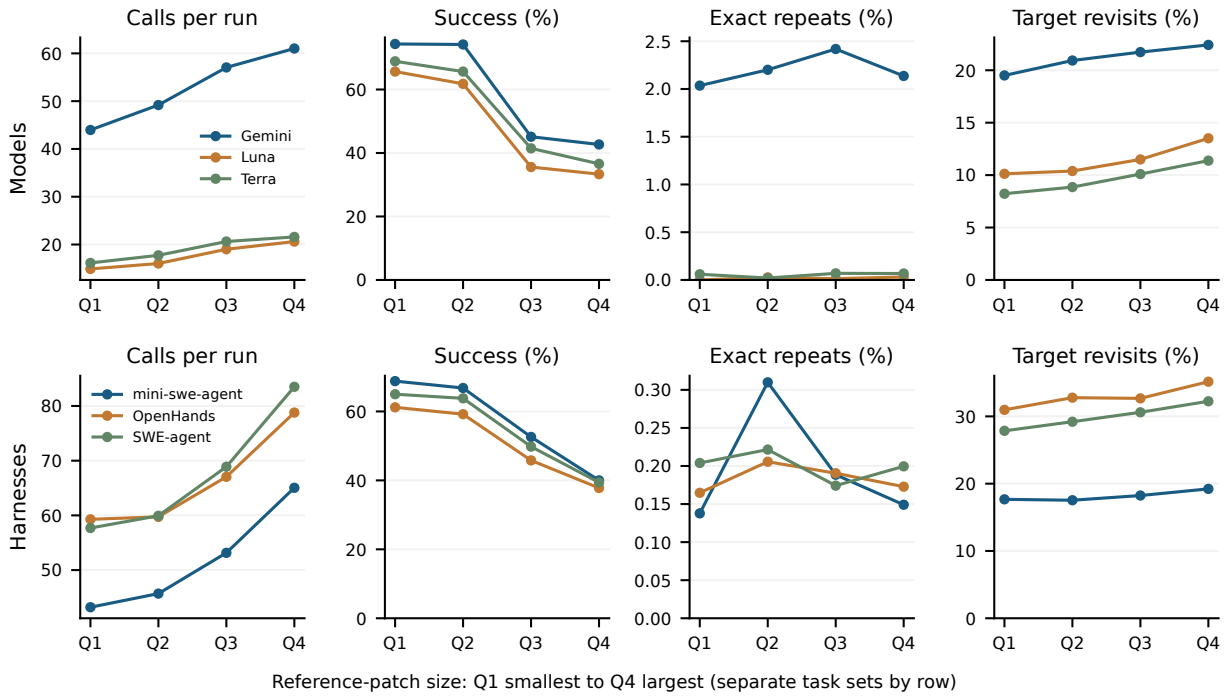


Figure 1: Restored patch-size analysis. Q1 contains the smallest reference patches and Q4 the largest. The model set has 657 tasks with patch metadata: 165 in Q1 and 164 in each remaining bin. Each task has nine runs. The harness set has 500 matched tasks per bin and three runs per task. All rates are means of per-run rates. Lines show descriptive means without uncertainty intervals. Task sets differ between rows.

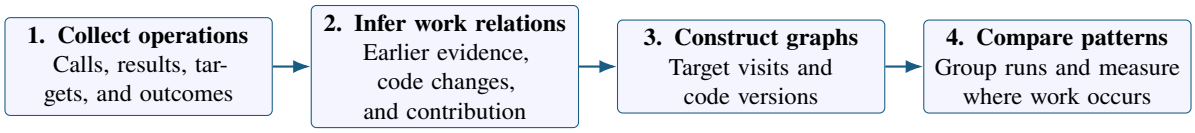


Figure 2: Four stages of WorkTrace. Operation IDs connect both graph views to the source records. The graphs summarize external actions. They do not reveal hidden model reasoning.

Pattern I: Final-version checking. Gemini performs more checks on its final inferred version than on an average earlier version. Figure 3 shows this pattern in the saved four-version groups. We test it numerically on the common subset defined in Section 3.4. For run r, we define the verification gap as

$$G_r = C_{r,\text{final}} - \frac{1}{|V_r^-|} \sum_{v \in V_r^-} C_{r,v}, \qquad (1)$$

where $C_{r,v}$ counts checks on version v. The set V_r^- contains other inferred versions and excludes the initial repository state. A positive gap means more checks on the final version.

Under the support-file exclusion rule, Gemini averages 3.15 final checks and 0.61 checks per earlier version. Luna averages 0.39 and 0.43. Terra averages 0.47 and 0.57. These values use the same 892 task–repeat pairs per model.

Gemini's mean gap exceeds Luna's by 2.58 checks [2.30, 2.84]. It exceeds Terra's by 2.64 checks [2.37, 2.92]. The direction remains under the other two version rules (Figure 4a). The Gemini–Luna contrast ranges from 1.80 to 2.58 checks across these rules. The gap also appears in both resolved and unresolved Gemini runs. Under support-file exclusion, their mean gaps are 2.46 [2.19, 2.73] and 2.48 [2.12, 2.85], respectively. These groups use all eligible Gemini runs, rather than the common subset. The association therefore does not require final task success.

What this explains. Gemini's additional execution work includes a final-version checking phase. Total call counts cannot locate that phase. The graph can. However, the final version can fail the task. Its position does not establish correctness or sufficient verification.

REVISION 08. Annotate property scope and relevant state for the verification pilot. Separate new coverage, repeated deterministic evidence, and justified stochastic reruns. Test scope saturation before claiming oververification.

Pattern II: Target revisits. With Qwen fixed, the harnesses differ in how often they return to known targets. Mean revisit rates are 18.2% for mini-swe-agent, 32.9% for OpenHands, and 30.0% for SWE-agent. These use one consistent per-run definition (Figure 4b).

Paired comparisons give an OpenHands–mini-swe-agent gap of 14.73 percentage points [14.26, 15.19]. The SWE-agent–mini-swe-agent gap is 11.81 points [11.37, 12.25]. Removing the task with 164 repeated probe executions changes these gaps to 14.76 and 11.85 points.

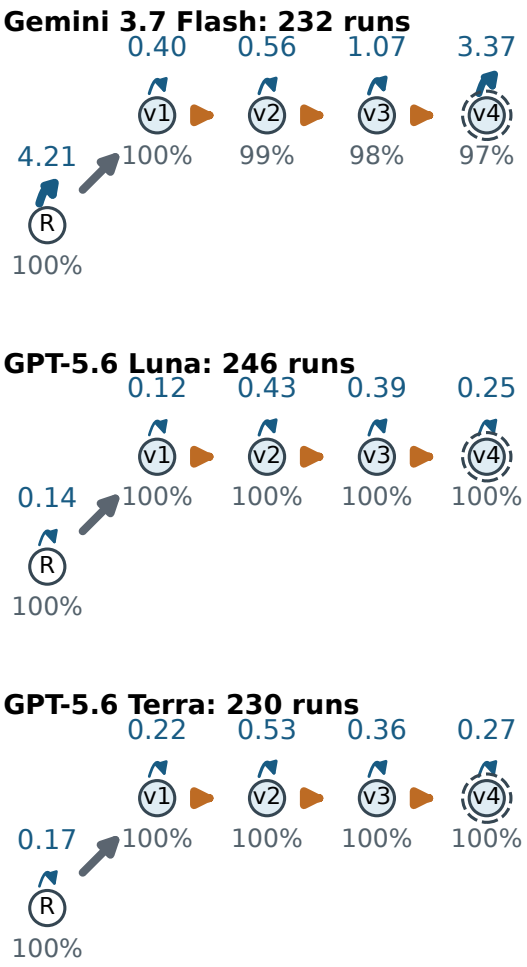


Figure 3: Saved four-version patterns. R is the initial repository state. Blue loops show mean checks at each coordinate. Percentages show node presence. Dashed circles mark frequently final nodes. Rare components are pruned. These graphs do not mark correctness.

The direction also holds within both source datasets (Appendix B).

Exact-repeat contrasts are much smaller. Relative to mini-swe-agent, the mean differences are −0.013 points for OpenHands and +0.004 points for SWE-agent. Their intervals include zero. This does not establish equivalence, but it shows that exact repeats alone miss the much larger revisit contrast.

What this explains. The harnesses organize recorded target access differently. They can revisit a file through a new command or a new range. Tool interfaces also change how the parser identifies targets. Thus, the pattern identifies a comparison that requires evidence inspection. It does not prove memory failure or excessive exploration.

REVISION 09. Normalize file paths, read ranges, and mixed calls. Recompute the paired revisit gaps. In the exploration pilot, separate new evidence from reacquisition for the same question.

5.2 Trace the Evidence

A graph shows where to inspect. Evidence about an operation shows what we can conclude from it.

Do later final-version checks use new commands? The saved Gemini reconciliation contains 6,699 final-version checks across 2,100 runs. A run's first check is automatically a new command within that version. We remove this first check from each nonempty run before assessing later checks.

The remaining 4,964 checks include 89.0% new commands [87.5, 90.5]. Identical command-and-output reruns account for 2.6% [2.1, 3.1]. Same-command checks with different outputs account for 8.4% [7.1, 9.8]. These rates pool checks and resample tasks.

Thus, most later checks are not exact reruns. But command variety does not establish new test coverage. Two pytest commands can select overlapping tests. Different tests can exercise the same property. This evidence supports a checking phase, while leaving its benefit unresolved.

REVISION 10. Recover full commands and test IDs before labeling scope. The scale reconciliation retains only 80-character command prefixes. Mark missing scope uncertain; report its frequency.

Do revisits add new information? The 50-run model pilot contains 1,432 operations. Of 225 operations with retrieved candidates, 51 receive a non-distinct label. Only 21 receive both reproduction and non-distinct labels. Those 21 are candidates for redundancy under the current evidence rules.

The 60-run Qwen pilot contains 4,018 operations. Non-distinct shares among operations with candidates are 5.1% for mini-swe-agent, 5.2% for OpenHands, and 9.7% for SWE-agent. These small pilot estimates cannot establish the meaning of every revisit in the 6,000-run set.

The important next distinction is whether a revisit addresses an unresolved question. A return can confirm a code change, inspect another file range, or recover missing context. Each case requires the operation and its earlier evidence. Graph shape alone cannot decide which case applies.

REVISION 11. Give reference annotators task instructions and trace prefixes, without model labels or rule facts. Allow any earlier evidence match. Preserve independent labels before adjudication.

5.3 Proposed Evidence Assessment

The current graphs locate checking phases and target revisits. Assessing excess requires a task-specific benefit criterion. We propose evidence links, informed by CoTJudger, and separate evidence novelty from task justification, following CAID (Li et al., 2026; Lee and Yu, 2026). This extension has not been implemented or evaluated.

Required annotation. For each assessed operation, record:

1. **Work unit:** the question or testable property, with a supporting trace span.
2. **State basis:** the relevant files, test selection, configuration, and their change evidence. Record unchanged, changed, or uncertain relative to the cited earlier operation.

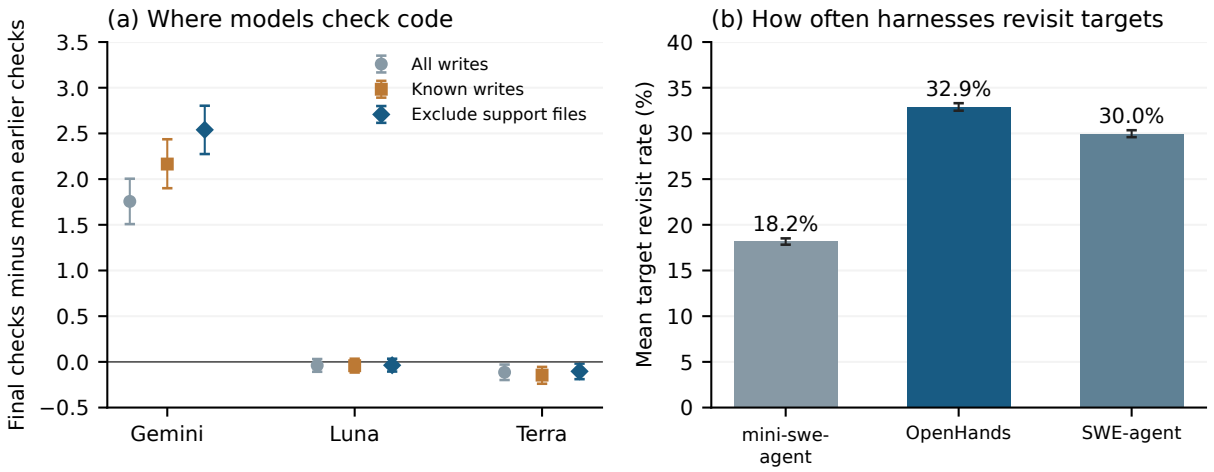


Figure 4: New analyses of saved data. (a) Verification gaps under three version rules on a fixed subset of 418 tasks. Each model has 892 runs. (b) Mean revisit rates on the same 2,000 tasks for each harness. Bars and points use 95% task-bootstrap intervals. Both plots describe the current measurements, before state-tracking repairs.

3. **Evidence gain:** new, repeated, or uncertain, with current and earlier operation IDs.
4. **Task justification:** justified, unsupported, or uncertain, with a reason based on the task and trace prefix.

Judge justification without the final patch or outcome. In a separate retrospective pass, link observations to visible decisions they support. Link relevant changes to earlier evidence they invalidate. Mark each link explicit or inferred. Record unclear support as unknown, not absent. Several observations can support one decision.

Verification rule. Flag a candidate when the same property receives repeated evidence on an unchanged relevant state. Review whether another check adds coverage or justified confidence. Unknown state or test scope requires an uncertain judgment. The same command after a relevant edit is a necessary control case.

Test *scope saturation* as an explanation of Pattern I. Within a fixed relevant state, plot cumulative distinct properties against check count. Report the fraction of assessable later checks that add no property. Report unknown scope separately. Repeated stochastic checks still require a confidence assessment.

Exploration rule. Flag a candidate when an operation repeats evidence for a settled question without a relevant change. Also assess novel information with no supported role in the task. A negative search that eliminates a plausible cause remains justified.

Test *evidence reacquisition* as an explanation of Pattern II. Report the fraction of assessable normalized returns that repeat available evidence for the same question on unchanged relevant state. Keep new file ranges and unresolved alternatives as controls. An absent support link or exclusion from the final patch does not establish excess.

REVISION 12. Run the pilot in Appendix D. Export operation IDs, prefix citations, state basis, gain, justification, and support links. Report changed diagnoses against the current graph view.

Claim boundary. CoTJudger's sufficient path is retrospective. It does not establish which actions were justified before the answer was known. Likewise, PACE's history compression does not establish removable tool execution. Independent review can support a low-benefit judgment under a stated criterion. A continuation experiment is required to test preserved success and execution-cost reduction.

REVISION 13. If testing removal, resume agents from matched prefixes and compare total cost and task success. Include a cost-matched control. Evaluate history compression separately and prevent future-patch leakage.

6 Conclusion

WorkTrace reconstructs how coding agents inspect targets, change code, and check versions. The current data shows two patterns: Gemini concentrates more checks on the final inferred version, and the Qwen harnesses differ in target revisits. New paired analyses preserve these differences under the tested measurement choices.

These patterns make system comparisons more specific. They identify where work accumulates and which evidence needs inspection. Independent validation and corrected state tracking remain necessary before using the framework to measure excessive work.

REVISION 14. Update the abstract only after the evidence pilot. Keep scope saturation and evidence reacquisition as hypotheses unless the independent judgments support them.

Limitations

Measurement. The audit reproduces errors in state identity, branch changes, mutation targets, restore outcomes, and incomplete diffs. These errors can change the version graphs. The new sensitivity analyses use existing measurements and cannot establish that the errors are harmless.

Validation. Automatic annotators agree on many labels, but the intended human reference remains incomplete. Candidate retrieval restricts the labels. The external CodeTraceBench result is unfavorable and appears in Appendix A.

Scope. Version matching selects runs with particular observed behavior. Target identifiers depend on tool interfaces. The two comparison sets use different tasks and released configurations. We therefore report system associations rather than isolated model or harness causes.

Utility. We do not observe intermediate patch correctness or the outcomes of stopping early. We also lack a complete monetary or time cost for every operation. Additional checks and revisits must not be counted as wasted work by default.

References

Xinliang Frederick Zhang, Anhad Mohananey, Alexandra Chronopoulou, Pinelopi Papalampidi, Somit Gupta, Tsendsuren Munkhdalai, Lu Wang, and Shyam Upadhyay. 2026. Do LLMs Really Need 10+ Thoughts for "Find the Time 1000 Days Later"? Towards Structural Understanding of LLM Overthinking. In *Proceedings of ACL*, pages 17005–17030. https://aclanthology.org/2026.acl-long.773/.

Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. https://www.software-lab.org/publications/ase2025_trajectories.pdf.

Liang Zhu, Bo Yang, Zhinuo Zhou, Jiachen Liang, Jiahao Guo, Yiyang Yin, Xiongwei Han, and Minyang Hu. 2026. Redundant or Necessary? A Benchmark for Detecting Redundant Steps in Agent Trajectories. arXiv:2605.29893v2. https://arxiv.org/abs/2605.29893v2.

Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing Cost of LLM Agents with Trajectory Reduction. arXiv:2509.23586v2. https://arxiv.org/abs/2509.23586v2.

Alex Mathai, Shobini Iyer, Aleksandr Nogikh, Petros Maniatis, Franjo Ivančić, Junfeng Yang, and Baishakhi Ray. 2026. TRIM: Reducing AI-Generated CodeSlop via Agent Trajectory Minimization. arXiv:2607.18161. https://arxiv.org/abs/2607.18161.

Igor Slinko, Yaroslav Golubev, and Sergey Titov. 2026. Opinion: Coding-Agent Benchmarks Should Match Their Users' Task Flows. arXiv:2610.09633v2. https://arxiv.org/abs/2610.09633v2.

NVIDIA. n.d. Open-SWE-Traces, v1.2 dataset release. https://huggingface.co/datasets/nvidia/Open-SWE-Traces.

NJU-LINK. n.d. CodeTraceBench, dataset release. https://huggingface.co/datasets/NJU-LINK/CodeTraceBench.

Siyi Li, Jiajun Shi, Shiwen Ni, Ge Zhang, Shuaimin Li, Shijian Wang, Zhoufutu Wen, Yizhi Li, Hamid Alinejad-Rokny, Jiaheng Liu, Min Yang, and Wenhao Huang. 2026. CoT-Judger: A Graph-Driven Framework for Automatic Evaluation of Chain-of-Thought Efficiency and Redundancy in LRMs. In *Findings of ACL*, pages 41837–41863. https://aclanthology.org/2026.findings-acl.2077/.

Daeyeop Lee and Hwanjo Yu. 2026. Valid ≠ Necessary: Diagnosing Latent Inefficiency in Chain-of-Thought. In *Findings of ACL*, pages 38998–39014. https://aclanthology.org/2026.findings-acl.1942/.

A Validation

Automatic annotation. The primary annotator is recorded as DeepSeek-V4.1-Flash through the configured endpoint. The JetBrains pilot has 50 runs from 17 tasks. The Qwen pilot has 60 runs from 20 tasks. Larger partial label files are excluded from these pilot results.

A secondary annotator, GLM-5.3-Flash, labels 225 matched items and 200 sampled unmatched items from the JetBrains pilot. Matched contribution agreement is 90.2%, with $\kappa = 0.70$. Matched relation agreement is 84.0%, with $\kappa = 0.50$. Candidate-redundancy agreement across 425 items has $\kappa = 0.83$. Three primary-model samples give contribution $\kappa = 0.88$–0.90 on matched items.

These checks measure agreement. They do not establish accuracy. Both annotators receive the same candidates and rule facts. The 200-item human packet has no adjudicated contribution labels.

External comparison. The saved CodeTraceBench analysis includes 484 trajectories, 17,317 steps, and 69 human-labeled unuseful steps (NJU-LINK, n.d.). Table 4 reports the result. Only 32 of the 69 human-labeled steps have a retrieved WorkTrace candidate.

Predictor	Predicted	Precision	Recall
Exact repeat	412	5.6%	33.3%
Target revisit	4,599	0.6%	39.1%
Candidate redundant	536	3.5%	27.5%

Table 4: Saved comparison against unuseful-step labels. These labels differ from WorkTrace's contribution construct. The result still provides no positive external validation.

Return after a detour. The saved pilot includes 153 returns after visits to other targets. Removing two uncertain contributions leaves 151 returns, 32 non-distinct labels, and 17 task clusters. A task-clustered GEE gives adjusted odds ratios of 32.7 [2.3, 456] for two to three intervening targets. The estimate for four or more is 14.5 [1.1, 193], relative to one target. The model controls run position, run length, and model identity. The intervals are wide, and the pattern mainly occurs in long runs. We retain this as a secondary exploratory result.

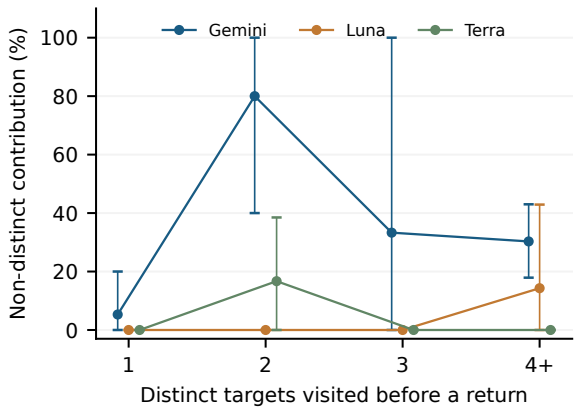


Figure 5: Restored return analysis using saved v2 labels. A detour counts distinct intervening targets. Points show non-distinct shares and saved task-bootstrap intervals. Bins contain 3–76 operations. One Luna interval cannot be estimated from the saved sample. The small, uneven groups do not establish a monotonic effect or excessive exploration.

B Additional Analyses

Version sensitivity. Table 5 uses the same 892 task–repeat pairs under all three rules. Every model has 892 runs from 418 tasks. Each contrast subtracts Luna's within-run verification gap from Gemini's gap.

Version rule	Difference	95% interval
All writes	1.80	[1.54, 2.05]
Known writes only	2.21	[1.94, 2.48]
Exclude support files	2.58	[2.30, 2.84]

Table 5: Paired Gemini–Luna contrasts in checks. Sensitivity to version rules changes the size but not the direction. All rules share unresolved implementation risks.

Source-dataset sensitivity. The matched harness set contains 1,063 Scale-SWE tasks and 937 SWE-rebench-V2 tasks. Table 6 compares each harness with mini-swe-agent within a source dataset.

Source	Harness	Gap	95% interval
Scale-SWE	OpenHands	16.60	[15.96, 17.23]
Scale-SWE	SWE-agent	11.80	[11.17, 12.39]
SWE-rebench	OpenHands	12.60	[11.93, 13.27]
SWE-rebench	SWE-agent	11.82	[11.21, 12.45]

Table 6: Paired differences in mean revisit rates, in percentage points. The direction holds within both sources under the current target definition.

Exploration and uncertainty. These are additional analyses of data already used to develop the framework. Their intervals describe sampling variation under the stored measurements. They do not include parser error, annotator error, or the effect of selecting findings after inspection. Fresh tasks are required for confirmation.

C Data and Reproduction

The manuscript's `artifact_manifest.json` records figure/table inputs and hashes. The analysis script writes tables and a manifest to `results/paper_revision_20261026/`. The README gives reproduction commands. Manifests retain package settings and full dataset revisions. Figures use saved graph outputs.

Some raw traces and caches are absent. Recover unclipped commands and observations before assessing missing evidence scope.

REVISION 15. Archive full observation inputs and mark truncation. Record hashes for dataset, parser, prompt, labels, and outputs. Retain operation IDs linking every judgment to its evidence.

D One-Week Evidence Pilot

Days 1–2: Repair and freeze. Pass the seven audit checks and regenerate affected summaries. Target 30 natural episodes: six for guide development and 24 for evaluation, with no shared tasks. Evaluation uses two verification and two exploration episodes per system condition. Freeze episode IDs before judging benefit. Retain uncertain cases and log missing-context exclusions.

Days 3–4: Build the reference. Two readers independently annotate the four fields in Section 5.3 from task instructions and trace prefixes. Hide automatic labels and final outcomes. Annotate support/invalidation links in a separate full-trace pass. Preserve independent labels before adjudication. Mark unrecoverable context uncertain.

Controlled contrasts. Test five paired distinctions: equivalent evidence under different command wording; identical commands after relevant changes; new properties on unchanged code; useful new-target exploration; and novel unjustified detours. Reviewers verify each pair's state and evidence. Report these diagnostic cases separately from natural episodes.

Days 5–6: Compare and ablate. Freeze prediction rules on development tasks. Compare exact repetition, target/version summaries, and evidence records against the hidden adjudicated reference. Ablate state conditioning. Report per-field agreement, uncertainty, and classification errors by role and system. Score support-link precision/recall on assessable adjudicated links; report unknown support separately. List diagnoses changed by state conditioning.

REVISION 16. Day 7: export repaired summaries, episode IDs, independent labels, adjudication, and baseline-error tables. Report pilot feasibility, not population prevalence. Defer the full support graph if context prevents reliable links. Reserve fresh tasks for confirmation.